`Assignment 4` Interpolation and Fitting Problems

Shuifan Wang `shepherd.wang@foxmail.com`

April 10, 2025

Problem 1

The length of the curve is 16.0075.

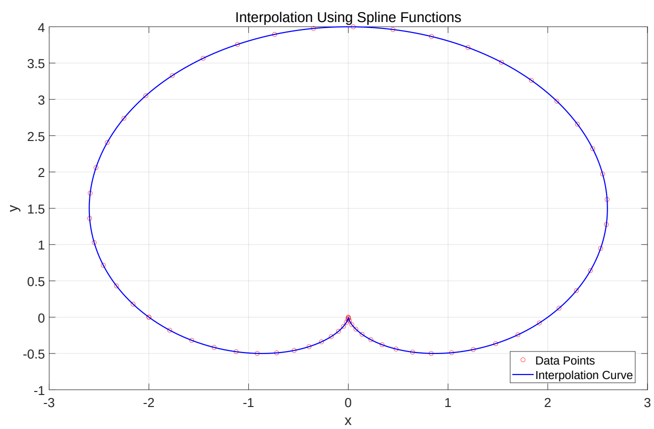

Figure 1: Problem 1

```
%-------------Problem 1------------
diary('output1.txt');
diary on;
clear; clc; close all;

```

```
%% Data Points
data = [
    -2.0000    -0.0000;
    -1.7913    -0.1797;
    -1.5707    -0.3184;
    -1.3460    -0.4164;
    -1.1248    -0.4755;
    -0.9137    -0.4992;
    -0.7186    -0.4916;
    -0.5442    -0.4584;
    -0.3938    -0.4055;
    -0.2694    -0.3395;
    -0.1713    -0.2668;
    -0.0987    -0.1939;
    -0.0493    -0.1267;
    -0.0195    -0.0702;
    -0.0049    -0.0287;
    -0.0004    -0.0050;
     0.0000    -0.0009;
     0.0021    -0.0165;
     0.0119    -0.0509;
     0.0347    -0.1016;
     0.0755    -0.1650;
     0.1381    -0.2362;
     0.2254    -0.3097;
     0.3389    -0.3793;
     0.4786    -0.4384;
     0.6434    -0.4806;
     0.8303    -0.4995;
     1.0354    -0.4895;
     1.2532    -0.4455;
     1.4773    -0.3640;
     1.7006    -0.2424;
     1.9152    -0.0797;
     2.1131     0.1236;
     2.2865     0.3653;
     2.4277     0.6417;
     2.5299     0.9477;
     2.5872     1.2767;
     2.5949     1.6211;
     2.5499     1.9725;
     2.4504     2.3216;
     2.2966     2.6591;
     2.0904     2.9757;
     1.8351     3.2624;
     1.5360     3.5111;
     1.1996     3.7143;
     0.8337     3.8662;
     0.4472     3.9622;
     0.0496     3.9995;
    -0.3493     3.9770;
    -0.7395     3.8953;
    -1.1113     3.7569;
    -1.4558     3.5659;
    -1.7649     3.3279;
    -2.0315     3.0499;
    -2.2503     2.7399;
```

```
    -2.4173     2.4067;
    -2.5303     2.0595;
    -2.5888     1.7079;
    -2.5939     1.3609;
    -2.5485     1.0273;
    -2.4569     0.7150;
    -2.3248     0.4307;
    -2.1587     0.1800
];

x = data(:, 1);
y = data(:, 2);

% Ensure the Curve is Closed
x = [x; x(1)];
y = [y; y(1)];

%% Parametrization
n = length(x);
t = zeros(n, 1);
for i = 2:n
    t(i) = t(i-1) + sqrt((x(i) - x(i-1))^2 + (y(i) - y(i-1))^2);
end

%% Interpolation Using Spline Functions
tt = linspace(t(1), t(end), 1000);
xs = spline(t, x, tt);
ys = spline(t, y, tt);

%% Calculate the Length
ppx = spline(t, x);
ppy = spline(t, y);

% Compute the First Derivative
dppx = fnder(ppx,1);
dppy = fnder(ppy,1);
dx = ppval(dppx, tt);
dy = ppval(dppy, tt);

% Compute Curve Length
ds = sqrt(dx.^2 + dy.^2);
curve_length = trapz(tt, ds);

%% Plot the Figure
figure;
plot(x, y, 'ro', 'MarkerSize', 6, 'DisplayName', 'Data Points');
hold on;
plot(xs, ys, 'b-', 'LineWidth', 1.5, 'DisplayName', ...
    'Interpolation Curve');
grid on;
xlabel('x');
ylabel('y');
title('Interpolation Using Spline Functions');
legend('Location', 'Best');

%% Print and Save
set(gca, 'FontSize', 18);
```

```
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'Problem1.pdf', 'ContentType', 'vector');

fprintf('Curve Length = %.4f\n', curve_length);

diary off;

% Output:
% Curve Length = 16.0075
```

Problem 2

$$rate = \frac{\beta_1 x_2 + \beta_2 x_3}{1 + \beta_3 x_1 + \beta_4 x_2 + \beta_5 x_3}$$
$$\beta_1 x_2 + \beta_2 x_3 - \beta_3 x_1 rate - \beta_4 x_2 rate - \beta_5 x_3 rate = rate$$

We may convert the model to a linear form like this. However, the result is not satisfactory, even if we use it as a start point to run the nonlinear regression, which means that the outcome may convert to this local optimum. The results are below. Nonlinear model with an arbitrary start point 0 fits the best.

Model	Fitted Results	RMSE
Linear Model	$\widehat{rate} = \frac{0.0015x_2 - 0.0034x_3}{1+0.0005x_1+0.0024x_2+0.0035x_3}$	20.7524
Nonlinear Model (AS)	$\widehat{rate} = \frac{0.1100x_2 - 0.0736x_3}{1+0.0049x_1+0.0082x_2-0.0162x_3}$	2.6946
Nonlinear Model	$\widehat{rate} = \frac{0.0025x_2 - 0.0024x_3}{1+0.0003x_1+0.0025x_2+0.0034x_3}$	5.3846

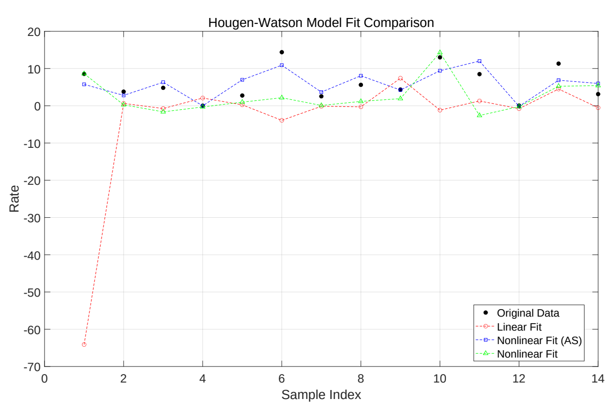

Figure 2: Problem 2

```
%-----------Problem 2------------
diary('output2.txt');
diary on;
clear; clc; close all;

%% Data Points
data = [470 300 10   8.55;
        285  80 10   3.79;
        470 300 120  4.82;
        470  80 120  0.02;
        100 190 10   2.75;
        100 190 120 14.39;
        100  80 65   2.54;
        100 190 65   5.62;
        470 190 65   4.35;
        100 300 54  13.00;
        100 300 120  8.50;
        100  80 120  0.05;
        285 300  10 11.32;
        285 190 120  3.13;
        ];

x = data(:, 1:3);
rate = data(:, 4);

%% Linear Fit
A = [x(:, 2), x(:, 3), -rate .* x(:, 1), -rate .* x(:, 2), -rate ...
    .* x(:, 3)];
B = rate;
beta_lin = A \ B;

disp('Linear Fit');
disp(beta_lin');

%% Nonlinear Fit (Arbitrary Start)
hougen = @(beta, x) (beta(1) * x(:, 2) + beta(2) * x(:, 3)) ./ ...
                        (1 + beta(3) * x(:, 1) + beta(4) * x(:, ...
                            2) + beta(5)*x(:, 3));
beta0 = [0 0 0 0 0];
opts = optimoptions('lsqcurvefit','Display','off');
beta_nlin1 = lsqcurvefit(hougen, beta0, x, rate, [], [], opts);

disp('Nonlinear Fit With Arbitrary Start');
disp(beta_nlin1);

%% Nonlinear Fit
hougen = @(beta, x) (beta(1)*x(:,2) + beta(2)*x(:,3)) ./ ...
                        (1 + beta(3)*x(:,1) + beta(4)*x(:,2) + ...
                            beta(5)*x(:,3));
beta0 = [0.0015 -0.0034 -0.0005 -0.0024 -0.0035];
opts = optimoptions('lsqcurvefit','Display','off');
beta_nlin2 = lsqcurvefit(hougen, beta0, x, rate, [], [], opts);

disp('Nonlinear Fit');
disp(beta_nlin2);

%% Plot the Figure
```

```
x1 = x(:,1);
x2 = x(:,2);
x3 = x(:,3);
rate_lin   = (beta_lin(1) * x(:, 1) + beta_lin(2) * x(:, 3)) ./ ...
    (1 + beta_lin(3) * x(:, 1) + beta_lin(4) * x(:, 2) + ...
    beta_lin(5) * x(: ,3));
rate_nlin1 = hougen(beta_nlin1, x);
rate_nlin2 = hougen(beta_nlin2, x);

figure;
plot(rate, 'ko', 'MarkerFaceColor','k', 'DisplayName','Original ...
    Data');
hold on;
plot(rate_lin, 'r--o', 'DisplayName','Linear Fit');
plot(rate_nlin1, 'b--s', 'DisplayName','Nonlinear Fit (AS)');
plot(rate_nlin2, 'g--^', 'DisplayName','Nonlinear Fit');

xlabel('Sample Index');
ylabel('Rate');
title('Hougen-Watson Model Fit Comparison');
legend('Location', 'Best');
grid on;

%% Compare RMSE
RMSE = @(y, yhat) sqrt(mean((y - yhat).^2));

RMSE_lin   = RMSE(rate, rate_lin);
RMSE_nlin1 = RMSE(rate, rate_nlin1);
RMSE_nlin2 = RMSE(rate, rate_nlin2);

fprintf('RMSE for Linear Fit: %.4f\n', RMSE_lin);
fprintf('RMSE for Nonlinear Fit With Arbitrary Start: %.4f\n', ...
    RMSE_nlin1);
fprintf('RMSE for Nonlinear Fit: %.4f\n', RMSE_nlin2);

%% Print and Save
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'Problem2.pdf', 'ContentType', 'vector');

diary off;

% Output:
% Linear Fit
%     0.0015   -0.0034   -0.0005   -0.0024   -0.0035
%
% Nonlinear Fit With Arbitrary Start
%     0.1100   -0.0736    0.0049    0.0082   -0.0162
%
% Nonlinear Fit
%     0.0025   -0.0024   -0.0003   -0.0025   -0.0034
%
% RMSE for Linear Fit: 20.7524
% RMSE for Nonlinear Fit With Arbitrary Start: 2.6946
% RMSE for Nonlinear Fit: 5.3846
```

Problem 3

Estimated Aircraft Position $(x, y, z) = (-60, 82, 10)$.

```
%------------Problem 3------------
diary('output3.txt');
diary on;
clear; clc; close all;

%% DME Station Data
DME_data = [
       0.0000,    10.0000,    0.2000,    94.2340,    0.4;
     200.0000,   300.0000,    0.5000,   339.4322,    0.2;
    -300.0000,   500.0000,    0.2000,   482.0996,    1.0;
    -400.0000,  -200.0000,    1.0000,   441.8201,    0.6;
     200.0000,  -200.0000,   -1.0000,   383.7252,    0.5
];

stations = DME_data(:, 1:3);
measured_distances = DME_data(:, 4);
errors = DME_data(:, 5);

%% Nonlinear Least Squares Estimate
residual = @(beta) ((sqrt(sum((stations - beta).^2, 2)) - ...
    measured_distances) ./ errors);
opts = optimoptions('lsqnonlin', 'Display', 'off');

num_trials = 2000;
initial_guesses = rand(num_trials, 3) * 1200 - 600;
best_beta = [];
best_resnorm = inf;

for i = 1:num_trials
    beta0 = initial_guesses(i, :);
    [beta_est, resnorm] = lsqnonlin(residual, beta0, [], [], opts);

    if resnorm < best_resnorm
        best_beta = beta_est;
        best_resnorm = resnorm;
    end
end

%% Display result
fprintf('Estimated Aircraft Position:\n');
fprintf('x = %.4f km\n', best_beta(1));
fprintf('y = %.4f km\n', best_beta(2));
fprintf('z = %.4f km\n', best_beta(3));
disp(['Minimum residual norm: ', num2str(best_resnorm)]);

diary off;

% Output:
% Estimated Aircraft Position:
% x = -60.0000 km
```

```
% y = 82.0001 km
% z = 9.9999 km
% Minimum residual norm: 5.3519e-09
```

Problem 4

Real signal ($n(t)$ is noise) :

$$x(t) = \sum_{k=1}^{P} (a_k \cos(\omega_k t) + b_k \sin(\omega_k t)) + n(t), \quad P = 2.$$

Results: $\omega_1 = 0.3974, a_1 = 5.0645, b_1 = 2.7412; \omega_2 = 0.6037, a_2 = 3.7723, b_2 = 2.2554.$

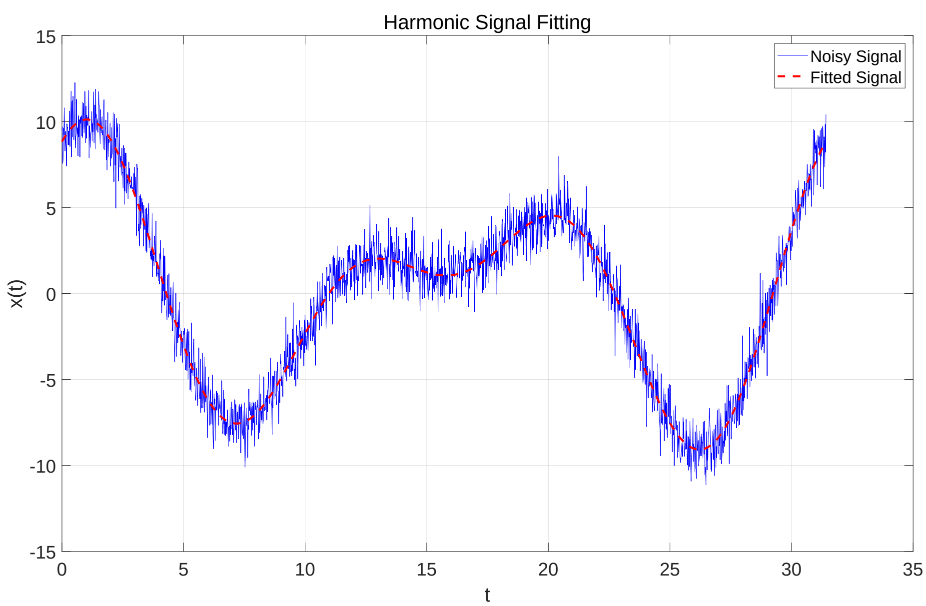

Figure 3: Problem 4

```
%-------------Problem 4-------------
diary('output4.txt');
diary on;
clear; clc; close all;

%% Load Data
```

```
data = readmatrix('data.xlsx');
t = data(:, 1);
x_obs = data(:, 2);

%% Nonlinear Least Squares
% params = [w1, w2, a1, a2, b1, b2]
harmonic_model = @(params, t) ...
    params(3) * cos(params(1) * t) + ...
    params(5) * sin(params(1) * t) + ...
    params(4) * cos(params(2) * t) + ...
    params(6) * sin(params(2) * t);
Fs = 1 / mean(diff(t));
L = length(t);

% With Clear Frequency Components, We Use FFT
Y = fft(x_obs);
P2 = abs(Y / L);
P1 = P2(1:floor(L/2));
f = Fs * (0:(L/2 - 1)) / L;
[¬, idx] = sort(P1, 'descend');

f1 = f(idx(1));
f2 = f(idx(2));
w1_guess = 2 * pi * f1;
w2_guess = 2 * pi * f2;
params0 = [w1_guess, w2_guess, 1, 1, 1, 1];

opts = optimoptions('lsqcurvefit', 'Display', 'iter');
[estimated_params, resnorm] = lsqcurvefit(harmonic_model, ...
    params0, t, x_obs, [], [], opts);

w1 = estimated_params(1);
w2 = estimated_params(2);
a1 = estimated_params(3);
a2 = estimated_params(4);
b1 = estimated_params(5);
b2 = estimated_params(6);

%% Plot Fitted Curve
x_fit = harmonic_model(estimated_params, t);

figure;
plot(t, x_obs, 'b-', 'DisplayName', 'Noisy Signal');
hold on;
plot(t, x_fit, 'r--', 'LineWidth', 2, 'DisplayName', 'Fitted ...
    Signal');
legend;
grid on;
xlabel('t'); ylabel('x(t)');
title('Harmonic Signal Fitting');

%% Print and Save
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'Problem4.pdf', 'ContentType', 'vector');

fprintf('Estimated parameters:\n');
```

```
fprintf('w1 = %.4f, a1 = %.4f, b1 = %.4f\n', w1, a1, b1);
fprintf('w2 = %.4f, a2 = %.4f, b2 = %.4f\n', w2, a2, b2);

diary off;

% Output:
% Estimated parameters:
% w1 = 0.3974, a1 = 5.0645, b1 = 2.7412
% w2 = 0.6037, a2 = 3.7723, b2 = 2.2554
```